

Slide 04:

Meus palpites são os seguintes:

1) Python: Creio que o operador "***" seja relativo a potência, então acredito que ele tenha a prioridade

R: 4, 8, 9

2) Java: Acredito que vai aparecer 339. Creio que ele fará a soma primeiro e a concatenação depois

3) JavaScript: Não faço ideia desse resultado, pois eu creio que JavaScript permite operações de números com strings, então não sei o que pensar

4) C: Nesse caso também não faço ideia do que vai aparecer como resultado

5) Python: Não faço ideia do que vai aparecer aqui

6) JavaScript: Acredito que o resultado será

false, false, false

Os gabaritos são:

1) -4512 -> Na primeira parte, temos -2^{**2} . Você poderia pensar que seria +4, contudo em Python a precedência do "***" é maior do que o "-" unário. Ou seja, ele faz a operação de 2 elevado a 2 e depois deixa o "-" no valor, portanto, -4. Quanto ao 512, eu me confundi no meu palpite.

Acontece que ele eleva $2^{**3^{**2}}$, ou seja é tudo junto, não separado como tinha imaginado. Para o Python, ele vai lendo da esquerda pra direita a ordem das potências, ou seja:

2 elevado a 3, este sendo elevado a 2 => $2^{(3^{**2})}$ => 512

2) 3345 -> Na primeira parte da expressão $(1 + 2)$, ele vai fazer soma, resultando em 3. Depois disso, tem $(+ "3")$. Nesse caso, o + é usado para concatenar, resultando em "33". Depois, como passamos a concatenar strings, os outros + vão seguir a mesma lógica $(+ 4 + 5)$. Resultando em "3345"

3) 352 -> Na primeira parte, temos uma subtração ("5" - 2). Como o - é usado para números, ele faz a subtração como se fosse um número, gerando 3. Na segunda parte temos o + ("5" + 2). Nesse caso, ele trata como concatenação, resultando em "52". No final, temos "352"

4) 0 -> Aqui a lógica está como Verdadeiro (1) e Falso (0). $3 > 2$ é verdadeiro, logo dá 1. Isso leva a próxima parte: $1 > 1$, que é falso, resultando no 0 que recebemos no terminal.

5) padrão True -> O que acontece é que 0 em Python é falso por padrão. Logo, em "x or padrão", quem sai é "padrão". Na segunda parte, temos $3 > 2 > 1$, o qual o Python interpreta como $3 > 2$ and $2 > 1$, que resulta em True. Então no final temos "padrão True"

6) true false true -> Na primeira comparação, o `==` converte a string "" em número. Ele vira 0, então a comparação fica $0 == 0$, o que é true. Na segunda comparação, ele compara duas strings diferentes, dá false. Na terceira é um caso clássico de operador de igualdade em JS. Mesmo um sendo string e o outro número, ele não diferencia os tipos, então dá true.

Slide 05:

Os tipos de precedência das operações dentro da linguagem podem variar.

Pense na expressão $a + b * c$, por exemplo.

Podemos fazer de diferentes formas:

1) Da esquerda pra direita: $(a + b) * c$

2) Da direita pra esquerda: $a + (b * c)$

Dependendo da expressão, posso ter resultados diferentes pela precedência.

Ex: $a = 3$, $b = 4$, $c = 5$

1) $(3 + 4) * 5 = 35$

2) $3 + (4 * 5) = 23$

Slide 06:

Temos diferentes expressões

- 1) $-2^{**}2$
- 2) $2^{**}3^{**}2$
- 3) $1 + 2 + "3"$
- 4) $-7 / 2$
- 5) $-7 \% 2$

Vamos como cada linguagem lida com essas contas.

Phyton:

- 1) $-4 \rightarrow$ Precedência do $**$ sobre o unário
- 2) 512
- 3) Erro $\rightarrow$ Não aceita $\text{int} + \text{str}$
- 4) $-4 \rightarrow$ com $//$, para divisão de inteiros
- 5) 1

JavaScript:

- 1) Erro $\rightarrow$ Ele não usa $**$ com número negativo
- 2) 512
- 3) "33" $\rightarrow$ Primeiro ele faz a soma dos números e depois concatena
- 4) -3.5
- 5) -1

Java:

- 1) Erro $\rightarrow$ não existe $**$ em Java
- 2) Erro $\rightarrow$ mesma razão
- 3) "33" $\rightarrow$ Primeiro ele soma enquanto tem só números. A partir do momento em que entram strings, ele concatena.
- 4) -3 $\rightarrow$ Ele acaba truncando a divisão pra baixo
- 5) -1

O slide também fala sobre Associatividade. Esse conceito fala sobre que direção uma expressão com operadores de mesma prioridade é calculada.

Por exemplo, em várias linguagens, $*$ e $/$ tem a mesma prioridade, então pela associatividade, saberemos em qual direção a expressão será executada

ESQUERDA $\rightarrow$ DIREITA

10 / 2 * 3

Agora pensando em **, no Python, a associatividade muda. Aí é da direita para a esquerda:

2 ** 3 ** 2 -> primeiro 3 ** 2 e depois 2 ** 9

Slide 07:

Esse slide fala sobre efeitos colaterais que funções podem exercer em expressões, dependendo da linguagem.

No exemplo, temos esse código aqui:

```
int a = 5;

int fun1(void) {
    a = 17;    /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

Dependendo da linguagem, o valor de a pode ser alterado para 17 na execução da função, resultando em 20 no final OU o a pode manter o seu valor 5 no final gerar 8. Depende se a função tem a capacidade de gerar esse efeito colateral ou não

EM JAVA: O resultado é 8, pois ele realmente vai ler a expressão da esquerda pra direita. Então "a" vai manter o valor 8.

EM C: O resultado vai ser 20, pois não há uma ordem definida por padrão como Java.

EM GO: O resultado dá 20, assim como C

Nos dois últimos casos, vemos que a função foi capaz de exercer um efeito colateral na expressão.

Slide 08:

Esse slide fala sobre a sobrecarga de operadores.

Sobrecarga de operadores é quando um mesmo operador pode ter diferentes significados dependendo dos tipos dos valores envolvidos.

Como exemplo, o operador + pode somar números (2 + 3), juntar strings ("Olá" + "mundo") ou até representar uma operação entre objetos, como vetores ou matrizes, em linguagens que permitem definir esse comportamento.

Isso pode deixar o código mais natural e legível, pois podemos escrever `v1 + v2` em vez de chamar uma função específica para somar dois vetores.

Contudo, a sobrecarga também pode dificultar a compreensão.

A sobrecarga pode ser definida pela própria linguagem ou pelo programador.

Slide 09:

Esse slide fala um pouco sobre conversões de tipos e os diferentes comportamentos.

Ocorrem acontecem quando um valor de um tipo é transformado em outro.

No primeiro caso temos o alargamento, ou seja, um tipo menor é convertido para um tipo maior, como `int` para `float`, sem grande risco de perda de informação.

EX: Em Java

```
int grande = 123456789;  
float f = grande;
```

No segundo, o estreitamento transforma um tipo maior em um menor e pode causar perda de dados, sendo muitas vezes necessário fazer um cast explícito.

Ex: Em Java

```
double x = 3.99;
```

```
int y = (int) x;
```

```
System.out.println(y); //3
```

Já a coerção ocorre quando a própria linguagem converte tipos automaticamente para permitir uma operação.

Ex: Em Java

```
int x = 5;  
double y = 2.5;  
  
double resultado = x + y;  
  
System.out.println(resultado); //7.5
```

Slide 10:

Coerção implícita: É quando o interpretador ou o compilador faz a troca de um tipo de uma variável em outra, sem que isso seja solicitado anteriormente. Isso normalmente ocorre em linguagens pouco tipadas. Como no slide de exemplo sendo o Javascript, um grande exemplo. Normalmente o PHP não é considerado uma linguagem fortemente tipada, mas tanto o PHP quanto o Javascript podem ocorrer a tipagem. Dentro do slide temos uma atividade para compararmos dois tipos de igualdades diferentes, vamos começar com o “===” do JavaScript, dentro da linguagem ele significa igualdade estrita, ou seja, a comparação não vai ser somente por aquilo que o dado representa, mas também pelo seu tipo, então vamos igualar o valor e seu tipo, isso nos dá uma segurança contra a coerção implícita que é feita pela linguagem. Agora o “!==”, pela linguagem PHP, ela significa o não identico, ou seja, ela não é igual ao “!==”, que compara somente o valor, nela a comparação é igual ao “===”, que falei anteriormente, ela compara tanto o tipo quanto o valor, servindo como uma barreira de proteção para a coerção implícita. Podemos verificar a partir dos exemplos que temos aqui uma escolha da linguagem, ou trade-off, onde essas linguagens escolheram a facilidade de escrita e velocidade em vez da confiabilidade.

Comentários do slide:

““b” + “a” + “a” + “a” = “baNaNa””: o +“a” é o mais unário aplicado a “a”, que dá NaN.

Regra prática em JavaScript e PHP: use === e !== (não fazem coerção). Linters como o ESLint (regra eqeqeq) obrigam isso.

PHP (type juggling): “0e123” == “0e456” é true porque os dois textos parecem números em notação científica e valem 0. Os hashes MD5 de “240610708” e “QNKCDZO” começam com “0e” seguidos só de dígitos. Um login que fizesse md5(\$senha) == \$hash_salvo aceitaria uma senha pela outra. O PHP 8 corrigiu parte disso: “abc” == 0 passou a ser false, mas “0e...” == “0e...” continua true.

Ligação com o Sebesta: o custo da coerção é a perda de detecção de erros (p. 312).

Linguagens de script escolheram facilidade de escrita em vez de confiabilidade.”

Demonstração:

JavaScript:

The screenshot shows the OneCompiler JavaScript editor. The code in the editor demonstrates various implicit coercion scenarios in JavaScript. The console output on the right shows the results of these operations, including comparisons between numbers, strings, booleans, and null/undefined values, as well as concatenation of strings and numbers.

```
1 // Aula 07 - Coerção implícita em JavaScript (Sebesta 7.4.1, p. 311)
2 // online: https://onecompiler.com/nodejs (cole o código inteiro; ou no console do navegador)
3 // local: node coercao.js
4 console.log("5" - 2); // 3 (o - só existe para números)
5 console.log("5" + 2); // 52 (o + prefere concatenar)
6 console.log("5" * "2"); // 10
7 console.log(true + 1); // 2
8 console.log([] + []); // (texto vazio)
9 console.log([] + {}); // [object Object]
10 console.log(null + 1, undefined + 1); // 1 NaN
11 console.log("b" + "a" + "a" + "a"); // ba11a11a
12
13 // == faz coerção antes de comparar; === não
14 console.log(0 == "", 0 == "0", "" == "0"); // true true false (não é transitivo!)
15 console.log(0 === "", 0 === "0"); // false false
16 console.log(Number("12px"), parseInt("12px")); // NaN 12
17
```

Comentários: Nesta compilação conseguimos ver claramente a parte do implícito, ou, de como a linguagem julga os tipos.

Php:

The screenshot shows the OneCompiler PHP editor. The code in the editor demonstrates various type juggling scenarios in PHP. The console output on the right shows the results of these operations, including comparisons between strings and numbers, and the use of md5 hashes.

```
1 <?php
2 // Aula 07 - Coerção em comparações: o "type juggling" do PHP (Sebesta 7.4.1, p. 311)
3 // online: https://onecompiler.com/php (cole o código inteiro)
4 // local: php coercao.php
5 var_dump("0e123" == "0e456"); // true! - as duas "parecem" números: 0 x 10^123 == 0 x 10^456
6 var_dump("0e123" === "0e456"); // false: === não converte
7 var_dump("1" == "01"); // true
8 var_dump(100 == "1e2"); // true
9 var_dump("abc" == 0); // false no PHP 8 (era true no PHP 7!)
10
11 // Ataque clássico: dois hashes MD5 diferentes que começam com "0e" seguido só de dígitos
12 var_dump(md5("240610708"));
13 var_dump(md5("QNKCDZO"));
14 var_dump(md5("240610708") == md5("QNKCDZO")); // true: um login que compara com == aceita a senha errada
```

Comentários: Nesta compilação da linguagem de php conseguimos verificar prontamente como a linguagem realiza a coerção implícita, e podemos ver claramente o “Type Juggling”, onde os dois valores “Parecem iguais” ao PHP por começarem com a mesma letra e número.

Slide 11:

Expressões relacionais e booleanas: Expressões relacionais e booleanas: É a forma como as linguagens avaliam e comparam valores lógicos e matemáticos, e como a ordem dessas operações afeta o resultado final. Um ponto importante é a precedência: os operadores relacionais, como o de maior ou menor, têm uma precedência menor que os aritméticos. Ou seja, em uma expressão como "a + 1 > 2 * b", a linguagem primeiro resolve a matemática para só depois comparar os resultados. Dentro do slide temos uma atividade de quiz para

compararmos como diferentes linguagens lidam com a mesma expressão: $3 > 2 > 1$. Em C, essa expressão nos dá um resultado incomum e falso. Isso ocorre porque o C avalia a conta da esquerda para a direita: ele primeiro faz $3 > 2$, o que é verdadeiro e resulta no número 1; em seguida, ele compara esse $1 > 1$, o que se torna falso. Já em Python, essa mesma expressão é encadeada automaticamente pela linguagem, significando $(3 > 2) \text{ and } (2 > 1)$, o que resulta em verdadeiro, igual fazemos na matemática escolar. Além disso, temos o conceito de "Truthiness", onde linguagens sem um controle rígido aceitam qualquer valor numa condição. Em Python, um 0 ou uma lista vazia contam como falso, mas uma string "0" ou uma lista com itens contam como verdadeiro. Linguagens modernas como Java ou C# exigem estritamente um booleano (true ou false) nessas verificações. Podemos verificar a partir desses exemplos mais uma escolha da linguagem, ou trade-off, onde o Python e o C clássico escolheram a flexibilidade e facilidade de escrita, enquanto Java e C# escolheram restringir isso em prol da confiabilidade e detecção precoce de erros.

Comentários do Slide: Tempo: ~3 min. DEMONSTRAÇÃO: relacionais.c e relacionais.py.

Sebesta (p. 313-314): os operadores relacionais têm precedência MENOR que os aritméticos ($a + 1 > 2 * b$ faz a aritmética primeiro). O resultado é booleano, exceto em C89, que não tinha tipo booleano: 0 é falso e qualquer outro valor é verdadeiro (p. 315).

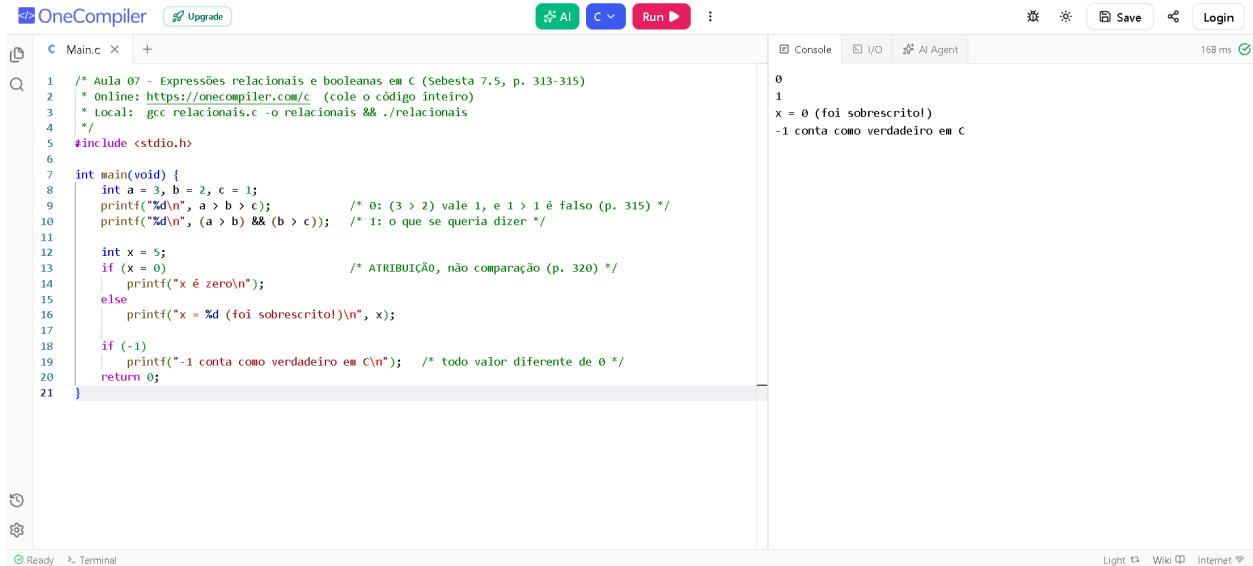
Resposta do quiz: em C, $3 > 2 > 1$ vale 0. O livro chama isso de "resultado incomum" (p. 315): $(3 > 2)$ dá 1, e $1 > 1$ é falso. A comparação entre b e c nunca acontece.

Em Python, a mesma expressão é ENCADEADA: $3 > 2 > 1$ significa $(3 > 2) \text{ and } (2 > 1)$, que é True, como na matemática.

"Truthiness": Python e JavaScript aceitam qualquer valor numa condição. 0, "", [], {} e None contam como falso, mas "0", [0] e " " contam como verdadeiro. Java, C#, Go e Rust EXIGEM um booleano no if, e o livro (p. 315) defende isso pela legibilidade e pela detecção de erros. Perl e Ruby têm && e 'and' com precedências diferentes (p. 315).

Demonstração

C:



The screenshot shows the OneCompiler C compiler interface. The code editor on the left contains a C program with the following content:

```
1 /* Aula 07 - Expressões relacionais e booleanas em C (Sebesta 7.5, p. 313-315)
2 * online: https://onecompiler.com/c (cole o código inteiro)
3 * local: gcc relacionais.c -o relacionais && ./relacionais
4 */
5 #include <stdio.h>
6
7 int main(void) {
8     int a = 3, b = 2, c = 1;
9     printf("%d\n", a > b > c); /* 0: (3 > 2) vale 1, e 1 > 1 é falso (p. 315) */
10    printf("%d\n", (a > b) && (b > c)); /* 1: o que se queria dizer */
11
12    int x = 5;
13    if (x = 0) /* ATRIBUIÇÃO, não comparação (p. 320) */
14        printf("x é zero\n");
15    else
16        printf("x = %d (foi sobrescrita)\n", x);
17
18    if (-1)
19        printf("-1 conta como verdadeiro em C\n"); /* todo valor diferente de 0 */
20    return 0;
21 }
```

The console on the right shows the output of the program:

```
0
1
x = 0 (foi sobrescrita!)
-1 conta como verdadeiro em C
```

Comentários:

Precedência: O compilador resolve primeiro as expressões matemáticas ($a + 1$ e $2 * b$) para só depois aplicar o operador relacional ($>$), retornando 1 (representação de verdadeiro em C). O "resultado incomum": A expressão $3 > 2 > 1$ imprime 0 (Falso). Isso ocorre porque a linguagem C avalia da esquerda para a direita. Primeiro ela resolve $(3 > 2)$, que retorna o número 1. Em seguida, ela faz $1 > 1$, o que é matematicamente falso, resultando em 0. A variável c nem precisaria existir, a lógica de encadeamento falha.

Tipagem (Falta de booleano estrito): Como o C89 não possuía o tipo `bool` nativo, qualquer número diferente de 0 (como o `-42` do exemplo) é aceito pelo `if` como verdadeiro, o que pode causar bugs difíceis de rastrear em comparação com linguagens mais modernas (como Java e Rust, que barrariam esse código).

Python:



The screenshot shows the OneCompiler Python compiler interface. The code editor on the left contains a Python program with the following content:

```
1 # Aula 07 - Expressões relacionais e booleanas em Python (Sebesta 7.5, p. 313-315)
2 # online: https://onecompiler.com/python (cole o código inteiro)
3 # local: python relacionais.py
4 a, b, c = 3, 2, 1
5 print(a > b > c) # True: Python encadeia -> (a > b) and (b > c)
6
7 # Valores que contam como FALSO ("falsy") em um if
8 for v in [0, 0.0, "", [], {}, None, "0", [0], " "]:
9     print(f"{v!r:6} -> {bool(v)}")
10
```

The console on the right shows the output of the program:

```
True
0 -> False
0.0 -> False
'' -> False
[] -> False
{} -> False
None -> False
'0' -> True
[0] -> True
' ' -> True
```

Comentários:

Encadeamento natural: Diferente do C, Python trata $3 > 2 > 1$ de forma natural e matemática. Internamente, o interpretador traduz isso para $(3 > 2)$ and $(2 > 1)$, o que resulta corretamente em True. Isso aumenta drasticamente a legibilidade (Capacidade de Leitura) da linguagem. Truthiness: Python é muito flexível com o que considera verdadeiro ou falso (truthiness). O código demonstra que o if aceita strings e listas. O comportamento padrão é: coleções ou strings vazias (`""`, `[]`) são consideradas False. Porém, se houver qualquer conteúdo dentro delas — até mesmo uma string com o número zero (`"0"`) ou uma lista com o zero dentro (`[0]`) —, Python avaliará a condição como True.

Slide 12:

Avaliação em curto-circuito: É quando o resultado de uma expressão lógica é determinado sem a necessidade de avaliar todos os seus operandos. Por exemplo, em uma expressão como $(a \geq 0) \&\& (b < 10)$, se $a < 0$, a primeira parte já é falsa. Como é uma operação "E" ($\&\&$), o resultado final será falso independentemente do valor de b. Portanto, a linguagem nem sequer avalia o segundo termo. Isso não apenas otimiza o código, mas também serve como um mecanismo de segurança essencial. Imagine uma busca em um array: `while ((index < listlen) && (list[index] != key))`. Se o índice ultrapassar o tamanho da lista, o primeiro termo falha, acionando o curto-circuito. Se a linguagem não fizesse isso, ela tentaria acessar `list[listlen]`, o que geraria um erro fatal de índice fora dos limites. O risco do curto-circuito ocorre quando o segundo operando possui um efeito colateral, como um incremento (`b++`). Se o curto-circuito for acionado, o `b++` nunca acontecerá. Linguagens baseadas em C utilizam $\&\&$ e $\|\|$ para curto-circuito, enquanto operadores como $\&$ e $\|$ (sem repetição) avaliam ambos os lados, mesmo com booleanos. Linguagens como Python, Ruby e Perl aplicam curto-circuito em todos os operadores lógicos. Modernamente, linguagens adotaram curto-circuitos especializados para lidar com nulos, como o encadeamento opcional (`?.`) e a coalescência nula (`??`). Um bug comum em JavaScript (e Python) ocorre ao usar o operador $\|\|$ (ou or) para definir valores padrão: a expressão `0 || 10` retornará 10, pois o 0 é avaliado como falso (truthiness), sobrescrevendo um valor que poderia ser válido. O operador `??` resolve isso focando exclusivamente em valores nulos.

Comentários do slide: “Tempo: ~4 min. DEMONSTRAÇÃO: CurtoCircuito.java e depois `curto_circuito.js`.

Definição (p. 315): o resultado é determinado sem avaliar todos os operandos. Em $(a \geq 0) \&\& (b < 10)$, se $a < 0$ o segundo termo é irrelevante.

Exemplo do livro (p. 316): a busca `while ((index < listlen) && (list[index] != key))`. SEM curto-circuito, quando a chave não está na lista, `list[listlen]` seria acessado e lançaria uma exceção de índice. O curto-circuito protege o acesso.

Linguagens baseadas em C: $\&\&$ e $\|\|$ fazem curto-circuito. $\&$ e $\|$ com booleanos NÃO fazem (p. 317). Ruby, Perl, ML, F# e Python fazem curto-circuito em todos os operadores lógicos (p. 317).

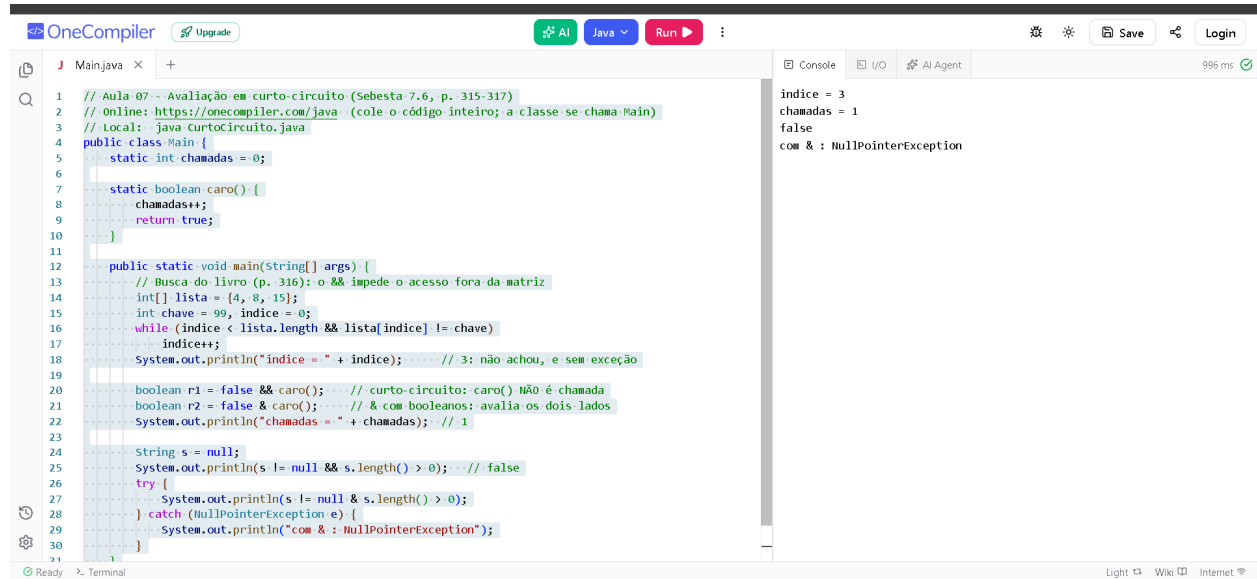
Risco (p. 316): curto-circuito combinado com efeito colateral. Em $(a > b) \|\| ((b++) / 3)$, o `b++` só acontece às vezes.

Modernos: o `?.` (encadeamento opcional) e o `??` (coalescência nula) de JavaScript, TypeScript, C# e Kotlin são curto-circuitos especializados em null.

Resposta do quiz (5): em Python, x or "padrão" devolve "padrão" quando x = 0. O or devolve um dos OPERANDOS, e o 0 conta como falso. Por isso quantidade || 10 é um bug em JS quando 0 é válido, e o ?? resolve."

Demonstração:

Java:



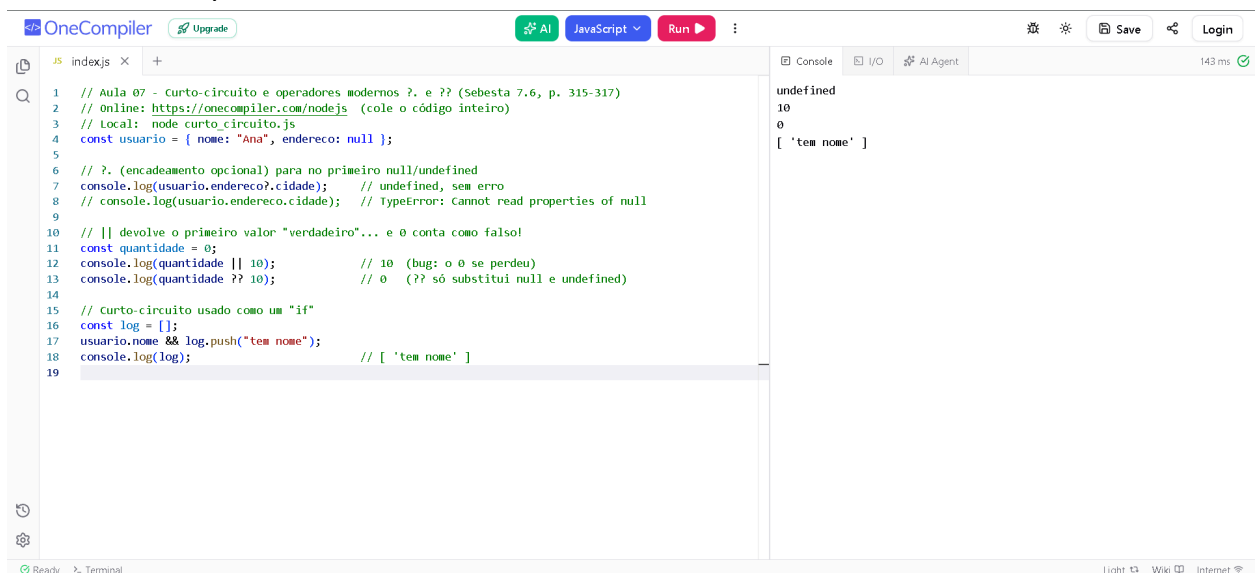
```
1 // Aula 07 - Avaliação em curto-circuito (Sebesta 7.6, p. 315-317)
2 // online: https://onecompiler.com/java (cole o código inteiro; a classe se chama Main)
3 // local: java CurtoCircuito.java
4 public class Main {
5     static int chamadas = 0;
6
7     static boolean caro() {
8         chamadas++;
9         return true;
10    }
11
12    public static void main(String[] args) {
13        // Busca do livro (p. 316): o && impede o acesso fora da matriz
14        int[] lista = {4, 8, 15};
15        int chave = 99, indice = 0;
16        while (indice < lista.length && lista[indice] != chave)
17            indice++;
18        System.out.println("indice = " + indice); // 3: não achou, e sem exceção
19
20        boolean r1 = false && caro(); // curto-circuito: caro() NÃO é chamada
21        boolean r2 = false & caro(); // & com booleanos: avalia os dois lados
22        System.out.println("chamadas = " + chamadas); // 1
23
24        String s = null;
25        System.out.println(s != null && s.length() > 0); // false
26        try {
27            System.out.println(s != null & s.length() > 0);
28        } catch (NullPointerException e) {
29            System.out.println("com & : NullPointerException");
30        }
31    }
32 }
```

Console output:

```
indice = 3
chamadas = 1
false
com & : NullPointerException
```

Comentários: Nesta demonstração em Java, podemos ver como o operador && protege contra exceções. No laço while, ele impede um erro de índice. O exemplo com a função caro() evidencia a diferença crucial: && faz o curto-circuito, interrompendo a avaliação no false, enquanto o & comercial avalia os dois lados incondicionalmente, mesmo que a primeira condição já determine o resultado. O último bloco prova como a falta do curto-circuito (&) causa um NullPointerException ao tentar checar o comprimento de uma string nula.

JavaScript:



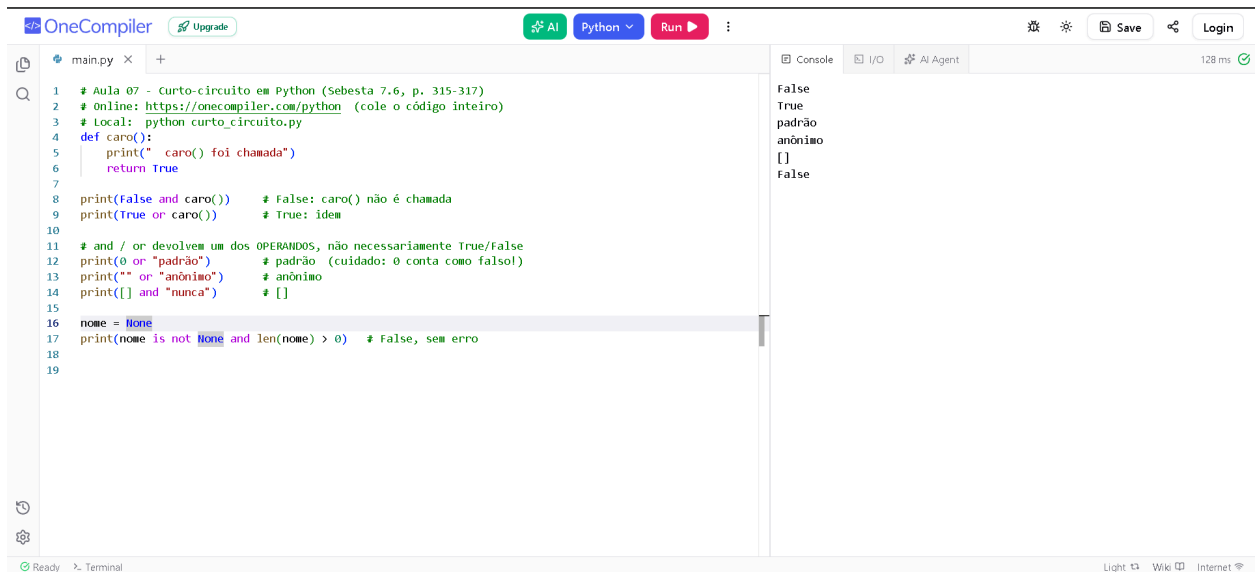
```
1 // Aula 07 - Curto-circuito e operadores modernos ?, e ?? (Sebesta 7.6, p. 315-317)
2 // online: https://onecompiler.com/nodejs (cole o código inteiro)
3 // local: node curto_circuito.js
4 const usuario = { nome: "Ana", endereco: null };
5
6 // ?. (encadeamento opcional) para no primeiro null/undefined
7 console.log(usuario.endereco?.cidade); // undefined, sem erro
8 // console.log(usuario.endereco.cidade); // TypeError: Cannot read properties of null
9
10 // || devolve o primeiro valor "verdadeiro"... e 0 conta como falso!
11 const quantidade = 0;
12 console.log(quantidade || 10); // 10 (bug: o 0 se perdeu)
13 console.log(quantidade ?? 10); // 0 (?? só substitui null e undefined)
14
15 // Curto-circuito usado como um "if"
16 const log = [];
17 usuario.nome && log.push("tem nome");
18 console.log(log); // [ 'tem nome' ]
19
```

Console output:

```
undefined
10
0
[ 'tem nome' ]
```

Comentários: O código JavaScript ilustra evoluções recentes para lidar com ausência de dados de forma segura. O operador `?.` faz um "curto-circuito" ao encontrar uma propriedade nula, evitando um `TypeError` que paralisaria o sistema. O código também demonstra claramente a falha ao usar `||` para valores padrão: se a quantidade for legitimamente zero, o `||` a descarta, assumindo o valor 10. O novo operador `??` soluciona isso. Por fim, vemos um truque comum: o uso do `&&` como um "if" em uma única linha.

Python:



```
1 # Aula 07 - Curto-circuito em Python (Sebesta 7.6, p. 315-317)
2 # Online: https://onecompiler.com/python (cole o código inteiro)
3 # Local: python curto_circuito.py
4 def caro():
5     print(" caro() foi chamada")
6     return True
7
8 print(False and caro()) # False: caro() não é chamada
9 print(True or caro())  # True: idem
10
11 # and / or devolve um dos OPERANDOS, não necessariamente True/False
12 print(0 or "padrão")   # padrão (cuidado: 0 conta como falso!)
13 print("" or "anônimo") # anônimo
14 print([] and "nunca")  # []
15
16 nome = None
17 print(nome is not None and len(nome) > 0) # False, sem erro
18
19
```

Comentários: Em Python, todos os operadores lógicos (`and`, `or`) possuem curto-circuito nativo. Como provado na função `caro()`, o segundo operando é completamente ignorado se a primeira parte da expressão já garantir o resultado. O mais importante é que `and` e `or` não devolvem valores booleanos estritos (`True` ou `False`), mas sim o último operando avaliado, demonstrando que `0 or "padrão"` retorna o texto "padrão", o que pode causar o mesmo bug exibido no JavaScript. O último exemplo mostra o curto-circuito atuando de forma idêntica ao Java para proteger uma variável nula (`None`) de causar erros durante verificações.

Slide 13:

Sentenças de atribuição: A atribuição é a operação mais fundamental em linguagens imperativas, responsável por alterar o estado de uma variável armazenada na memória. A forma simples mais adotada pelas linguagens é o `=`, embora o ALGOL utilizasse `:=` justamente para evitar confusão com o operador matemático de igualdade. Além da forma simples, temos a atribuição composta (como `a += b`, herdada do ALGOL 68 e popularizada pelo C) e a atribuição unária (`count++`), que condensa a operação em um único operando. Um conceito muito importante discutido pelo Sebesta é a atribuição agindo como uma EXPRESSÃO. Em linguagens como C, JavaScript e Perl, quando você atribui um valor (`a = 5`), essa operação inteira devolve o próprio valor 5. Isso permite encadeamentos do tipo `sum = count = 0` (lido da direita para a esquerda) ou construções complexas dentro de laços `while`. O custo dessa flexibilidade é a confiabilidade: em C, é muito comum cometer o erro de digitar `if (x = y)` em vez

de `if (x == y)`; o compilador aceita, a atribuição ocorre no meio do `if`, e a condição é avaliada com base no valor atribuído, causando bugs terríveis. Linguagens mais seguras, como Java e C#, bloquearam isso exigindo que o `if` receba estritamente um valor booleano. Por outro lado, linguagens modernas como Python e Go trouxeram a "atribuição múltipla", permitindo trocar valores de duas variáveis com um simples `a, b = b, a`, sem precisar de uma terceira variável temporária. O Python, historicamente, proibiu que a atribuição fosse uma expressão para evitar o bug do `if (x = y)`. No entanto, na versão 3.8, foi introduzido o "operador morsa" (`:=`), permitindo atribuir e testar o valor na mesma linha, de forma controlada.

Comentários do slide: "Tempo: ~5 min. DEMONSTRAÇÃO: `atribuicao.c`, `atribuicao.py` e `atribuicao.go`."

Formas (p. 317-321):

- Simples: quase todas as linguagens usam `=`. ALGOL usou `:=` para não confundir com a igualdade.
 - Composta: `a += b` (vem do ALGOL 68 e foi adotada por C).
 - Unária: `count++` e `++count`. Em `count++`, o incremento acontece antes da negação (p. 319).
 - Como EXPRESSÃO (p. 319-320): em C, JavaScript e Perl, a atribuição devolve o valor atribuído. Isso permite `while ((ch = getchar()) != EOF)` e `sum = count = 0`. O exemplo do livro `a = b + (c = d / b) - 1` aparece no arquivo.
 - O custo (p. 320): `if (x = y)` em vez de `if (x == y)` compila em C. Java e C# evitam o problema porque o `if` só aceita booleano.
 - Múltipla (p. 320-321): Perl, Ruby, Lua, Python e Go permitem `a, b = b, a` (troca sem variável temporária).
 - Funcional (p. 321): em ML e F#, `val` e `let` dão NOME a um valor, que nunca muda. O `let` de Rust é imutável por padrão, e `"let x = x + 1"` cria uma NOVA variável (sombreamento).
- Python: a atribuição NÃO é expressão, exceto pelo operador morsa `:=` (3.8), criado justamente para casos como o `while` do `getchar`. Go: `x++` é sentença, e `y := x++` não compila."

Demonstração:

C:



```
1 /* Aula 07 - Atribuição como expressão em C (Sebesta 7.7, p. 317-321)
2  * Online: https://onecompiler.com/c (cole o código inteiro)
3  * Local: gcc atribuicao.c -o atribuicao && ./atribuicao
4  */
5 #include <stdio.h>
6
7 int main(void) {
8     int sum, count;
9     sum = count = 0; /* múltiplos alvos: a atribuição devolve um valor (p. 320) */
10    printf("sum %d\n", sum, count);
11
12    int a, b = 10, c, d = 30;
13    a = b + (c = d / b) - 1; /* exemplo do livro (p. 320): c = 3, a = 12 */
14    printf("a = %d, c = %d\n", a, c);
15
16    const char *texto = "abc";
17    int i = 0;
18    char ch;
19    while ((ch = texto[i++]) != '\0') /* padrão clássico de C (p. 319) */
20        printf("[%c]", ch);
21    printf("\n");
22
23    count += 5; /* atribuição composta (p. 318) */
24    count++; /* atribuição unária (p. 318) */
25    printf("count = %d\n", count); /* 6 */
26    return 0;
27 }
28
```

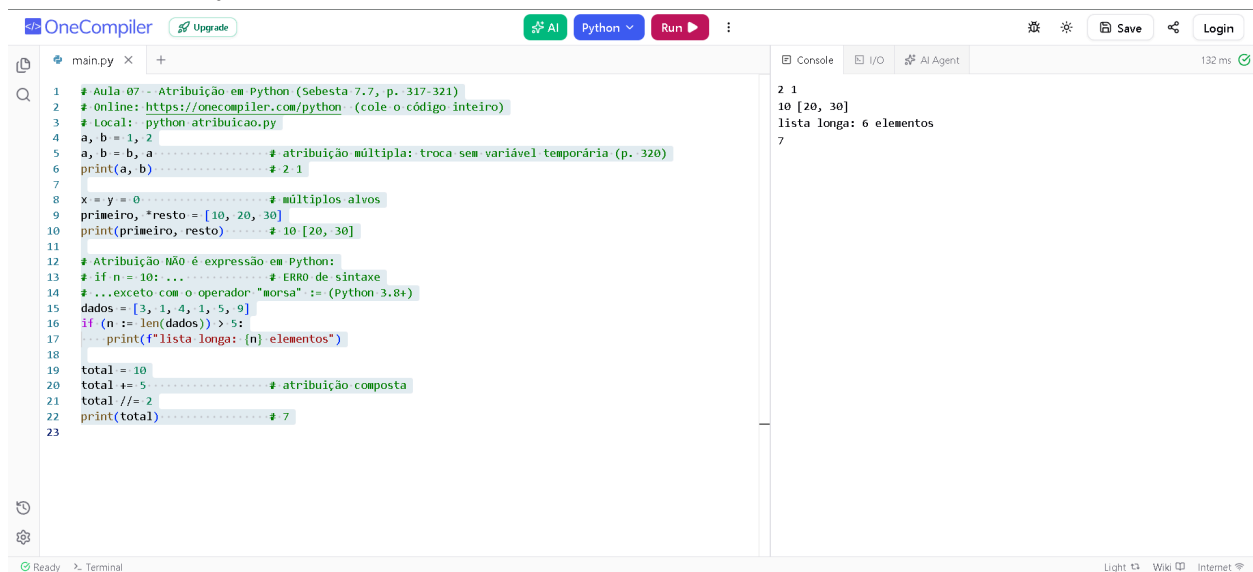
Console Output:

```
0 0
a = 12, c = 3
[a][b][c]
count = 6
```

Comentários: O código em C ilustra perfeitamente a atribuição funcionando como

uma expressão que retorna valor. Isso é evidenciado na linha `sum = count = 0`, onde a avaliação da direita para a esquerda atribui 0 a `count`, que por sua vez "devolve" 0 para ser atribuído a `sum`. O "exemplo do livro" (`a = b + (c = d / b) - 1`) mostra como isso pode deixar o código terrivelmente confuso (baixa legibilidade) ao embutir uma atribuição matemática no meio de outra expressão. Por fim, vemos o padrão clássico do C no laço `while`, onde a variável `ch` recebe o caractere `e`, simultaneamente, o `while` avalia se esse caractere recém-atribuído é diferente do fim da string (`'\0'`).

Python:



```
1 # Aula 07 - Atribuição em Python (Sebesta 7.7, p. 317-321)
2 # online: https://onecompiler.com/python (cole o código inteiro)
3 # local: python atribuiçao.py
4 a, b = 1, 2
5 a, b = b, a ..... # atribuição múltipla: troca sem variável temporária (p. 320)
6 print(a, b) ..... # 2 1
7
8 x = y = 0 ..... # múltiplos alvos
9 primeiro, *resto = [10, 20, 30]
10 print(primeiro, resto) ..... # 10 [20, 30]
11
12 # Atribuição NÃO é expressão em Python:
13 # if n = 10: ..... # ERRO de sintaxe
14 # ...exceto com o operador "morsa" := (Python 3.8+)
15 dados = [3, 1, 4, 1, 5, 9]
16 if (n := len(dados)) > 5:
17     print(f"lista longa: {n} elementos")
18
19 total = 10
20 total += 5 ..... # atribuição composta
21 total /= 2
22 print(total) ..... # 7
23
```

Console output:

```
2 1
10 [20, 30]
lista longa: 6 elementos
7
```

Comentários: O script em Python destaca a conveniência da "atribuição múltipla", permitindo inverter valores na mesma linha (`a, b = b, a`) de forma muito limpa e legível. Mostra também o recurso de desempacotamento de listas usando o asterisco (`*resto`). O ponto crucial dessa demonstração é a proteção do Python contra a atribuição acidental em condições: tentar usar dentro de um `if` gera um erro de sintaxe imediato. Para contornar isso e ainda permitir atribuições atreladas a verificações lógicas, o Python exige o uso do operador `morsa` (`:=`), como visto no teste do comprimento da lista (`n := len(dados)`).

Slide 14:

Atribuição de modo misto: Atribuição de modo misto: Ocorre quando atribuímos um valor de um tipo a uma variável de outro tipo. A grande questão de projeto discutida pelo Sebesta é: a expressão precisa ter exatamente o mesmo tipo de variável, ou a linguagem deve forçar uma coerção (conversão automática)? Linguagens mais antigas como C, C++ e Perl permitem a coerção em qualquer direção, na maioria das vezes de forma totalmente silenciosa. Isso significa que você pode colocar um número quebrado (3.7) dentro de um inteiro, e a linguagem simplesmente corta a parte decimal sem avisar. Java e C# decidiram aumentar a confiabilidade proibindo metade dessas conversões: elas só permitem a coerção de **ALARGAMENTO**, ou seja, você pode colocar um `int` dentro de um `double` (pois não há perda de dados), mas o contrário gera um erro de compilação imediato. Em linguagens puramente funcionais, não existe atribuição de modo misto de forma alguma. Porém, há uma pegadinha clássica em Java com atribuições compostas: se você fizer `int i = 3;` e depois `i += 3.7;`, o código vai compilar perfeitamente! Isso acontece porque a especificação do Java define que

operadores como += possuem um cast implícito embutido. Conectando com nossos exercícios anteriores: ao fazermos $\text{double taxa} = \text{acertos} / \text{total} * 100$, se acertos e total forem inteiros, a divisão será puramente inteira ($7 / 10 = 0$), e só DEPOIS esse 0 será alargado para 0.0 para caber no double, gerando um erro lógico.

Comentário do slide: “Tempo: ~3 min. DEMONSTRAÇÃO: ModoMisto.java e modo_misto.c.

Questão de projeto (p. 321): a expressão precisa ter o mesmo tipo da variável que recebe o valor, ou vale coerção?

- C, C++ e Perl: coerção em qualquer direção. $\text{int } i = 3.7$ vira 3 em silêncio, e $\text{char } c = 300$ vira 44 (o gcc só avisar).

- Java e C# (p. 322): só permitem a coerção de ALARGAMENTO (int para float, mas não o contrário). Segundo o livro, proibir metade das atribuições de modo misto é um jeito simples e eficaz de aumentar a confiabilidade.

- Linguagens funcionais: não existe atribuição de modo misto.

Pegadinha do Java: $i += 3.7$ COMPILA, porque a especificação define que $E1 \text{ op} = E2$ equivale a $E1 = (T)((E1) \text{ op } (E2))$, com um cast implícito.

Conecte com o exercício (questão 6): $\text{double taxa} = \text{acertos} / \text{total} * 100$ dá 0.0. A divisão é INTEIRA ($7 / 10 = 0$) e só DEPOIS o resultado é alargado para double. ”

Demonstração

Java:

```
1 // Aula 07 - Atribuição de modo misto em Java (Sebesta 7.8, p. 321-322)
2 // Online: https://onecompiler.com/java (cole o código inteiro; a classe se chama Main)
3 // Local: java ModoMisto.java
4 public class Main {
5     public static void main(String[] args) {
6         double d = 3; // ok: alargamento int -> double
7         // int i = 3.7; // ERRO de compilação: estreitamento exige cast (p. 322)
8         int i = (int) 3.7; // 3
9         long l = 1; // ok: alargamento
10        // float g = 1.5; // ERRO: o literal 1.5 é double
11        char c = 65; // ok: a constante cabe em char
12        System.out.println(d + " * i + " + l + " + " + c); // 3.0 3 3 A
13
14        i += 3.7; // COMPILA! += inclui um cast: i = (int) (i + 3.7)
15        System.out.println(i); // 6
16    }
17 }
```

Console: 3.0 3 3 A
6

Comentários: Nesta compilação em Java, verificamos a regra de proteção (alargamento) em ação: tentar colocar 3.7 direto no int foi barrado pelo compilador para evitar perda de dados, exigindo que o programador force a barra escrevendo o (int) explicitamente. No entanto, o trecho final do código prova a pegadinha da linguagem: a operação composta $i += 3.7$ passa ilesa pela segurança do compilador porque embute secretamente o cast explícito que era exigido na atribuição simples, truncando silenciosamente a casa decimal e resultando em 6.

C:



```
1 /* Aula 07 - Atribuição de modo misto em C (Sebesta 7.8, p. 321-322)
2 * Online: https://onecompiler.com/c (cole o código inteiro)
3 * Local: gcc modo_misto.c -o modo_misto && ./modo_misto
4 */
5 #include <stdio.h>
6
7 int main(void) {
8     int i = 3.7; /* C converte em silêncio: trunca para 3 */
9     char c = 300; /* não cabe: sobra 44 (no máximo um aviso) */
10    unsigned int u = -1; /* 4294967295 */
11    printf("%d %d %u\n", i, c, u);
12    return 0;
13 }
14
```

Console

```
Main.c: In function 'main':
Main.c:9:14: warning: unsigned conversion from 'int' to 'char' changes
value from '300' to '44' [-Woverflow]
9 |     char c = 300; /* não cabe: sobra 44 (no máximo um
  |               ^
  |               aviso) */
3 44 4294967295
```

Comentários: A execução do código em C expõe o risco da conversão silenciosa em qualquer direção (estreitamento). A atribuição `int i = 3.7;` simplesmente joga a casa decimal no lixo sem dar nenhum alerta. O problema escala com `char c = 300;`: como um `char` suporta no máximo o valor 255, o número transborda (overflow) e o compilador `gcc` emite apenas um pequeno aviso não-bloqueante (warning), armazenando o lixo matemático 44 na variável, o que destruiria a confiabilidade do software em produção.

Slide 15:

Estrutura de controle: Fazendo agora a transição do Capítulo 7 para o 8, saímos da simples atribuição de valores para o fluxo do programa em si. Como o livro de Sebesta aponta, computações imperativas avaliam e atribuem, mas programas úteis precisam de dois mecanismos essenciais: escolher caminhos (seleção) e repetir ações (iteração). Como podemos ver na imagem, em 1966, Böhm e Jacopini provaram matematicamente que qualquer algoritmo existente pode ser escrito usando apenas três coisas: uma sequência, um "if-else" e um laço "while". Isso provou que o famoso e caótico "goto" era totalmente desnecessário, dando origem à programação estruturada. Mas se apenas três comandos bastam, por que linguagens modernas têm `for`, `switch`, `do-while`, `foreach`? A resposta é a facilidade de escrita. Criar estruturas específicas poupa tempo do desenvolvedor. No entanto, o limite para essa criação de estruturas é a legibilidade: se uma linguagem se torna gigante, os programadores acabam aprendendo subconjuntos diferentes dela, tornando o código de um ilegível para o outro. Por fim, definimos uma estrutura de controle como a sentença em si (o "if") mais o bloco que ela controla. A regra de ouro do design moderno é: as estruturas possuem apenas uma porta de entrada. Múltiplas entradas exigiram o retorno do famigerado "goto" e seus rótulos, destruindo a organização lógica do código.

Comentário do slide: "Tempo: ~2 min. Transição do Cap. 7 para o Cap. 8.

Sebesta (p. 328): as computações imperativas avaliam expressões e atribuem valores, mas programas úteis precisam de mais dois mecanismos: SELECIONAR entre caminhos e REPETIR sentenças.

Böhm e Jacopini (1966) provaram que qualquer algoritmo representável em fluxograma pode ser escrito só com sequência, seleção de dois caminhos e laço com pré-teste. O goto é desnecessário. Esse resultado apoiou a programação estruturada.

Por que as linguagens têm tantas estruturas a mais (for, switch, do-while, foreach...)? Pela facilidade de escrita (p. 329). O limite é a legibilidade: poucas pessoas aprendem todas as sentenças de uma linguagem grande, e cada leitor conhece um subconjunto diferente.

Definição (p. 330): uma estrutura de controle é uma sentença de controle mais as sentenças cuja execução ela controla. Questão de projeto geral: a estrutura pode ter MÚLTIPLAS ENTRADAS? (Hoje, uma entrada só, porque múltiplas entradas exigiram goto e rótulos.)”

Demonstração:

Comparação Conceitual: Linguagens Ricas (C/Java) vs. Linguagens Minimalistas (Go)

C: Linguagens como C, C++, Java e C# oferecem um arsenal vasto de estruturas de controle. Se você quer repetir algo, pode usar "while", "do-while", o "for" clássico (com inicialização, condição e incremento) ou o "foreach" (para iterar sobre coleções). Isso dá ao programador a ferramenta perfeita para cada micro-situação, maximizando a velocidade e a facilidade de escrita. Porém, como Sebesta alerta, isso exige que quem lê o código saiba exatamente como cada uma dessas estruturas se comporta (por exemplo, lembrar que o "do-while" executa pelo menos uma vez antes de testar a condição).

Go: A linguagem Go, criada pelo Google, levou a teoria de Böhm e Jacopini ao pé da letra para resolver o problema de "subconjuntos de linguagem" citado pelo Sebesta. Em Go, NÃO EXISTE "while", "do-while" ou "foreach". Existe apenas uma única estrutura de repetição: o "for". Se você quer um loop infinito, escreve apenas "for {}". Se quer um while, escreva "for (condição)". Se quer o clássico, escreve "for i:=0; i<10; i++". Os projetistas da linguagem sacrificaram propositalmente a criação de palavras-chave novas (reduzindo a facilidade de escrita) para garantir 100% de legibilidade: qualquer programador Go no mundo lê e escreve laços exatamente da mesma forma.

Slide 16:

Else pendente: É quando existem comandos de seleção aninhados com mais "if" do que "else", gerando a dúvida de a qual "if" o "else" pertence. Em C, a regra é que o "else" pertence ao "if" mais próximo que ainda não tem "else", e a endentação é ignorada pelo compilador, o que pode enganar o programador, como no exemplo do slide, que imprime 1 em vez de -1. Para associar o "else" ao "if" externo, é necessário usar chaves. Já em Python não existe essa ambiguidade, pois a endentação faz parte da sintaxe e define a qual "if" o "else" pertence. Em Rust o problema também não ocorre, porque as chaves são obrigatórias em todo "if" e "else".

Seletores que devolvem valor: São seleções que produzem um valor que pode ser atribuído diretamente a uma variável. Em C, o "if" é apenas um comando, então é usado o operador ternário (?:). Em Python existe a expressão condicional (valor if condição else valor). Já em Rust o próprio "if" é uma expressão, ou seja, o bloco escolhido devolve um valor, sendo obrigatório o uso do "else" e os dois ramos devendo ter o mesmo tipo.

Demonstração:

C:

```
C Main.c x +
1 #include <stdio.h>
2
3 int main(void) {
4     int sum = 0, count = 5, result = -1;
5
6     if (sum == 0)
7     {
8         if (count == 0)
9             result = 0;
10        else
11            result = 1;
12        printf("Sem chaves: %d\n", result);    /* 1 */
13
14        result = -1;
15        if (sum == 0) {
16            if (count == 0)
17                result = 0;
18        } else {
19            result = 1;
20        }
21        printf("Com chaves: %d\n", result);    /* -1 */
22
23        int x = -3;
24        int y = x > 0 ? x : 2 * x;
25        printf("Operador ternario: %d\n", y); /* -6 */
26    }
}
```

Sem chaves: 1
Com chaves: -1
Operador ternario: -6

Comentário: Nesta execução vemos que o primeiro trecho, sem chaves, imprime 1, mostrando que o "else" foi associado ao "if" mais próximo, mesmo com a endentação sugerindo o contrário. Com as chaves, o "else" passa a pertencer ao "if" externo e o resultado é -1. Na última linha, o operador ternário devolve -6, mostrando como C faz uma seleção que produz valor.

Python:

```
main.py x +
1 sum_, count, result = 0, 5, -1
2
3 if sum_ == 0:
4     if count == 0:
5         result = 0
6     else:
7         result = 1
8     print("Else do if externo:", result) # -1
9
10 result = -1
11 if sum_ == 0:
12     if count == 0:
13         result = 0
14     else:
15         result = 1
16     print("Else do if interno:", result) # 1
17
18 x = -3
19 y = x if x > 0 else 2 * x
20 print("Expressao condicional:", y) # -6
```

Else do if externo: -1
Else do if interno: 1
Expressao condicional: -6

Comentário: Nesta execução vemos que a endentação define a qual "if" o "else" pertence: alinhado ao "if" externo, o resultado é -1, e alinhado ao interno, é 1, sem nenhuma ambiguidade. A expressão condicional devolve -6, o mesmo valor obtido em C com o operador ternário.

Rust:

```

1 fn main() {
2     let sum = 0;
3     let count = 5;
4     let mut result = -1;
5
6     if sum == 0 {
7         if count == 0 {
8             result = 0;
9         }
10    } else {
11        result = 1;
12    }
13    println!("Else do if externo: {}", result); // -1
14
15    let x = -3;
16    let y = if x > 0 { x } else { 2 * x };
17    println!("if como expressao: {}", y); // -6
18 }
```

Console

I/O

AI Agent

Else do if externo: -1
if como expressao: -6

Comentário: Nesta execução o resultado é -1, pois as chaves obrigatórias deixam explícito que o "else" pertence ao "if" externo, eliminando o problema. Na segunda linha, o próprio "if" é usado como expressão e devolve -6 diretamente para a variável y.

Slide 17:

Seleção múltipla: É quando o programa escolhe um entre vários caminhos de acordo com o valor de uma variável. Em C, o "switch" trata cada "case" apenas como um ponto de entrada, e a execução continua pelos casos seguintes até encontrar um "break", o que é chamado de "cair" (fallthrough). No exemplo do slide, com index igual a 1, são executados odd, even e a mensagem de erro, pois não há nenhum "break". Esse comportamento permite agrupar valores, mas, por ser o padrão, o esquecimento de um "break" gera erros silenciosos.

As linguagens mais recentes corrigiram esse problema de formas diferentes. Em Go, cada caso termina sozinho, e para cair no próximo é preciso escrever "fallthrough" explicitamente. Em C#, cair implicitamente é proibido e gera erro de compilação. No Java 14 em diante, o "switch" com seta (->) não cai, permite agrupar valores com vírgula e pode ser usado como expressão, devolvendo um valor; além disso, é exaustivo para enum, ou seja, o compilador acusa erro se algum valor não for tratado. Em Rust, o "match" compara o valor com padrões, como valores exatos, faixas (1..=9) e o curinga "", *que funciona como o "default". O "match" também é exaustivo: o compilador verifica se todos os valores possíveis estão cobertos, e por isso, sem o "", o exemplo do slide não compila.* O Python 3.10 também ganhou o "match", com padrões e guardas (condições extras em cada caso).

Evidências:

a) C (switch.c):

```
C Main.c x +
1 #include <stdio.h>
2
3 int main(void) {
4     int index = 1, odd = 0, even = 0;
5
6     switch (index) {
7         case 1:
8             case 3: odd += 1;
9             case 2:
10             case 4: even += 1;
11             default: printf("Error\n");
12         }
13     printf("Sem break: odd=%d even=%d\n", odd, even);
14
15     odd = 0; even = 0;
16     switch (index) {
17         case 1:
18             case 3: odd += 1; break;
19             case 2:
20             case 4: even += 1; break;
21             default: printf("Error\n");
22         }
23     printf("Com break: odd=%d even=%d\n", odd, even);
24     return 0;
25 }
```

Console I/O AI Agent

Error
Sem break: odd=1 even=1
Com break: odd=1 even=0

Comentários: Nesta execução vemos o efeito do "cair": sem "break", o valor 1 incrementou odd, incrementou even e ainda imprimiu "Error". Com o "break", apenas odd foi incrementado, que era o comportamento esperado.

b) Java (Switch.java):

```
J Main.java x +
1 public class Switch {
2     enum Dia { SEG, TER, QUA, QUI, SEX, SAB, DOM }
3
4     public static void main(String[] args) {
5         for (Dia dia : Dia.values()) {
6             String tipo = switch (dia) {
7                 case SAB, DOM -> "fim de semana";
8                 case SEG, TER, QUA, QUI, SEX -> "dia util";
9             };
10            System.out.println(dia + ": " + tipo);
11        }
12    }
13 }
```

Console I/O AI Agent

SEG: dia util
TER: dia util
QUA: dia util
QUI: dia util
SEX: dia util
SAB: fim de semana
DOM: fim de semana

Comentário: Nesta execução o "switch" com seta classifica cada dia sem precisar de "break" e sem cair para os outros casos. O resultado do "switch" é atribuído diretamente à variável tipo, mostrando o switch como expressão, e como todos os dias do enum foram tratados, não foi necessário o "default".

c) Go (switch.go):

```
main.go X
1 package main
2
3 import "fmt"
4
5 func main() {
6     index := 1
7     odd, even := 0, 0
8
9     switch index {
10    case 1, 3:
11        odd++
12    case 2, 4:
13        even++
14    default:
15        fmt.Println("Error")
16    }
17    fmt.Printf("Sem fallthrough: odd=%d even=%d\n", odd, even)
18
19    odd, even = 0, 0
20    switch index {
21    case 1, 3:
22        odd++
23        fallthrough
24    case 2, 4:
25        even++
26    default:
27        fmt.Println("Error")
28    }
29    fmt.Printf("Com fallthrough: odd=%d even=%d\n", odd, even)
30 }
```

Console I/O AI Agent

Sem fallthrough: odd=1 even=0
Com fallthrough: odd=1 even=1

Comentário: Na primeira parte, cada caso termina sozinho e apenas odd é incrementado, sem precisar de "break". Na segunda, o "fallthrough" escrito explicitamente faz a execução cair no caso seguinte, incrementando também even, mas sem chegar ao "default". Isso mostra que em Go cair é uma escolha do programador e vale só para o próximo caso.

d) Rust (selecao.rs):

```
HelloWorld.rs X
1 fn main() {
2     for n in [-7, 0, 5, 42] {
3         let tipo = match n {
4             i32::MIN..=-1 => "negativo",
5             0 => "zero",
6             1..=9 => "um digito",
7             _ => "varios digitos",
8         };
9         println!("{}", n, tipo);
10    }
11 }
```

Console I/O AI Agent

-7: negativo
0: zero
5: um digito
42: varios digitos

Comentário: Nesta execução o "match" classifica cada número usando padrões: a faixa dos negativos, o valor exato 0, a faixa de 1 a 9 e o curinga "_" para o restante. Cada número entra em exatamente um caso, e o valor devolvido pelo "match" é atribuído diretamente à variável tipo.

Slide 18:

Laços controlados por contador: São laços que repetem um bloco de código usando uma variável que conta as repetições. Em C, o "for" é formado por três partes (inicialização,

condição e incremento), mas na prática funciona como um "while", pois a condição é testada a cada volta com o valor atual da variável. Por isso ele é chamado de laço lógico disfarçado. A variável de controle é uma variável comum, e alterá-la dentro do corpo muda a contagem: no exemplo do slide, o "i += 2" faz o laço imprimir apenas 0 3 6 9. Além disso, as três partes do "for" são opcionais, e um "for" só com a condição equivale a um "while".

Em Python, o "for" não possui condição nem incremento, ele percorre um iterável, entregando o próximo valor a cada volta. O "range(0, 10)" gera a sequência de 0 a 9 uma única vez no início, e por isso alterar o "i" dentro do corpo não muda nada: na volta seguinte o valor é sobrescrito pelo próximo da sequência, e o resultado continua sendo de 0 a 9. Quanto às questões de projeto, em C a variável declarada no "for" só existe dentro do laço e a condição é avaliada a cada volta, enquanto em Python a variável continua existindo depois do laço com o último valor recebido e o "range" é avaliado apenas uma vez. Assim, o mesmo código produz resultados diferentes nas duas linguagens.

Evidências:

a) C (laco_for.c):

```
C Main.c × +
1 #include <stdio.h>
2
3 int main(void) {
4     for (int i = 0; i < 10; i++) {
5         printf("%d ", i);
6         i += 2;
7     }
8     printf("\n");
9
10    int i = 1, soma = 0;
11    for (; i <= 100; )
12        soma += i++;
13    printf("Soma de 1 a 100: %d\n", soma);
14    return 0;
15 }
```

Console I/O AI Agent

0 3 6 9
Soma de 1 a 100: 5050

Comentário: Nesta execução vemos que alterar o "i" dentro do corpo muda a contagem, imprimindo apenas 0 3 6 9, pois a condição é testada a cada volta com o valor atual da variável. A segunda parte mostra o "for" só com a condição, funcionando como um "while" e somando os números de 1 a 100.

b) Python (laco_for.py):

```
main.py × +
1 for i in range(0, 10):
2     print(i, end=" ")
3     i += 2
4 print()
5 print("Valor de i depois do laço:", i)
6
7 for i in range(10, 0, -3):
8     print(i, end=" ")
9 print()
```

Console I/O AI Agent

0 1 2 3 4 5 6 7 8 9
Valor de i depois do laço: 11
10 7 4 1

Comentário: Nesta execução o laço imprime de 0 a 9 normalmente, mostrando que o "i += 2" não afetou a iteração, já que o "for" entrega o próximo valor do "range" a cada volta. A linha seguinte mostra que o "i" continua existindo depois do laço, com o valor 11, e a última mostra o "range" com passo negativo, gerando 10 7 4 1.

Slide 19:

Laços lógicos, saídas e iteradores: Os laços lógicos são controlados por uma condição, e não por um contador, e por isso precisam garantir que haja progresso a cada volta para que a condição se torne falsa em algum momento. Um exemplo real de falha nisso foi o bug do Zune, em 2008: quando a variável dias valia exatamente 366 em um ano bissexto, nenhum dos caminhos do laço alterava as variáveis, e o laço se repetia infinitamente, travando os aparelhos no último dia do ano. Também é importante diferenciar os laços de pré-teste, como o "while", que testam a condição antes de executar o corpo, dos laços de pós-teste, como o "do-while", que executam o corpo ao menos uma vez antes de testar, o que é fácil de esquecer.

Quanto às saídas, um "break" comum sai apenas do laço mais interno. O Java permite dar nomes (rótulos) aos laços, e com o "break" rotulado é possível sair de vários laços aninhados de uma vez, como no exemplo do slide, que interrompe a busca na matriz ao encontrar o primeiro número negativo. Já em relação aos iteradores, o JavaScript possui duas formas parecidas com comportamentos diferentes: o "for...in" percorre as chaves (os índices), que são entregues como texto, e por isso a soma acaba concatenando e resulta em "0012"; enquanto o "for...of" percorre os valores, resultando corretamente em 60.

Evidências:

a) C (zune.c):



```
1 #include <stdio.h>
2
3 int bissexto(int ano) {
4     return (ano % 4 == 0 && ano % 100 != 0) || ano % 400 == 0;
5 }
6
7 int main(void) {
8     int dias = 366, ano = 2008, voltas = 0;
9
10    while (dias > 365) {
11        if (++voltas > 1000) {
12            printf("Laco infinito detectado: dias=%d ano=%d\n", dias, ano);
13            break;
14        }
15        if (bissexto(ano)) {
16            if (dias > 366) { dias -= 366; ano++; }
17        } else { dias -= 365; ano++; }
18    }
19
20    int x = 10;
21    while (x < 5) printf("while executou\n");
22    do { printf("do-while executou uma vez\n"); } while (x < 5);
23    return 0;
24 }
```

Console: Laco infinito detectado: dias=366 ano=2008
do-while executou uma vez

Comentário: Nesta execução vemos o bug do Zune reproduzido: com dias igual a 366 no ano de 2008, o laço não fez progresso, pois nenhuma variável foi alterada, e só terminou porque um contador de segurança detectou o laço infinito, mostrando os valores travados. Na segunda

parte, o "while" não executou nenhuma vez, porque a condição já começava falsa, enquanto o "do-while" executou uma vez, mostrando a diferença entre pré-teste e pós-teste.

b) Java (Rotulos.java):



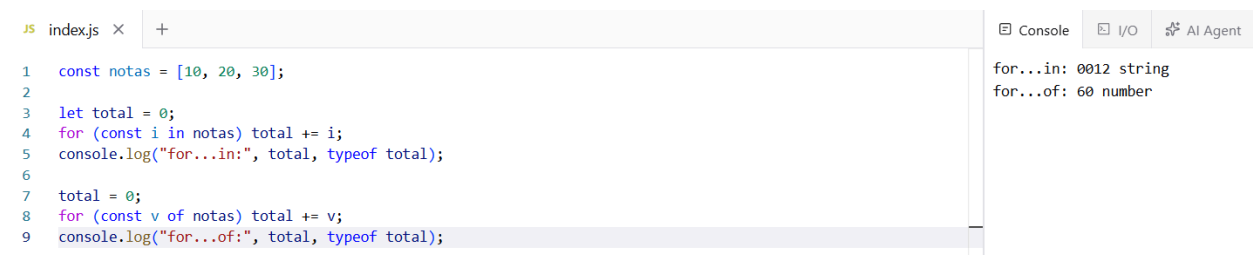
```
1 public class Rotulos {
2     public static void main(String[] args) {
3         int[][] m = { { 1, 2, -3},
4                       {-4, 5, 6} };
5
6         System.out.println("Com break externo:");
7         externo:
8         for (int i = 0; i < m.length; i++)
9             for (int j = 0; j < m[i].length; j++)
10                 if (m[i][j] < 0) {
11                     System.out.println(i + ", " + j);
12                     break externo;
13                 }
14
15         System.out.println("Com break comum:");
16         for (int i = 0; i < m.length; i++)
17             for (int j = 0; j < m[i].length; j++)
18                 if (m[i][j] < 0) {
19                     System.out.println(i + ", " + j);
20                     break;
21                 }
22     }
23 }
```

Console

```
Com break externo:
0,2
Com break comum:
0,2
1,0
```

Comentário: Nesta execução, com o "break externo", o programa encontrou o primeiro negativo da matriz, na posição 0,2, e saiu dos dois laços, sem verificar a linha seguinte. Com o "break" comum, ele saiu apenas do laço interno, continuou na linha 1 e encontrou também o negativo da posição 1,0, mostrando que o rótulo muda de qual laço o programa sai.

c) JavaScript (iteracao.js):



```
1 const notas = [10, 20, 30];
2
3 let total = 0;
4 for (const i in notas) total += i;
5 console.log("for...in:", total, typeof total);
6
7 total = 0;
8 for (const v of notas) total += v;
9 console.log("for...of:", total, typeof total);
```

Console

```
for...in: 0012 string
for...of: 60 number
```

Comentário: Nesta execução, o "for...in" percorreu os índices como texto, e a soma virou a concatenação "0012", do tipo string, por causa da coerção implícita. Já o "for...of" percorreu os valores do array, resultando em 60, do tipo number, que era o resultado esperado.

Slide 20:

Desvio incondicional: O "goto" é um comando que faz o programa pular diretamente para um ponto marcado com um rótulo, sem nenhuma condição. Em 1968, Dijkstra publicou a carta "Go To Statement Considered Harmful", argumentando que o "goto" torna o fluxo do programa difícil de acompanhar, o que impulsionou a programação estruturada. Hoje, C, C++, C# e Go ainda possuem "goto", enquanto Java não possui, apesar de manter "goto" como

palavra reservada. Para os casos mais comuns, as linguagens oferecem desvios restritos, como "break", "continue" e "return", que só podem ir para destinos bem definidos e por isso melhoram a legibilidade.

Um exemplo real dos riscos foi o bug "goto fail", encontrado no código de segurança da Apple em 2014. A função deveria verificar o hash e depois a assinatura do certificado, mas uma linha "goto fail" foi duplicada. Como o "if" não tinha chaves, apenas o primeiro "goto" pertencia a ele, e o segundo era executado sempre, apesar da indentação sugerir o contrário. Assim, quando o hash estava correto, a função pulava para o final com o valor 0, indicando "certificado válido", sem nunca verificar a assinatura, o que permitia aceitar certificados falsificados. O bug teria sido evitado com chaves obrigatórias, como em Go e Rust, ou compilando com "gcc -Wall", que avisa sobre indentação enganosa.

Evidências:

a) C (goto_fail.c):

<pre>1 #include <stdio.h> 2 3 int verifica_hash(void) { return 0; } /* hash correto */ 4 int verifica_assinatura(void) { return -1; } /* assinatura falsa */ 5 6 int valida_certificado(void) { 7 int err; 8 if ((err = verifica_hash()) != 0) 9 goto fail; 10 goto fail; 11 if ((err = verifica_assinatura()) != 0) 12 goto fail; 13 fail: 14 return err; 15 } 16 17 int valida_corrigido(void) { 18 int err; 19 if ((err = verifica_hash()) != 0) { 20 goto fail; 21 } 22 if ((err = verifica_assinatura()) != 0) { 23 goto fail; 24 } 25 fail: 26 return err; 27 } 28 29 int main(void) { 30 int r1 = valida_certificado(); 31 int r2 = valida_corrigido(); 32 printf("Com o bug: %d (%s)\n", r1, r1 == 0 ? "certificado valido" : "certificado invalido"); 33 printf("Corrigido: %d (%s)\n", r2, r2 == 0 ? "certificado valido" : "certificado invalido"); 34 return 0; 35 }</pre>	<pre>Com o bug: 0 (certificado valido) Corrigido: -1 (certificado invalido)</pre>
---	--

Comentários: Nesta execução simulamos um certificado com hash correto e assinatura falsa. A versão com o bug devolveu 0, indicando "certificado válido", pois o "goto fail" duplicado desviou a execução antes da verificação da assinatura. Na versão corrigida, com chaves em cada "if", a assinatura foi verificada e a função devolveu o erro corretamente, rejeitando o certificado.

Slide 21:

Comandos protegidos: São estruturas de seleção e repetição propostas por Dijkstra, nas quais cada comando é protegido por uma condição, chamada de guarda, e só pode ser executado se

ela for verdadeira. Diferente do "if" comum, em que as condições são testadas em ordem e a primeira verdadeira vence, no "if ... fi" todas as guardas são avaliadas. Se mais de uma for verdadeira, uma delas é escolhida de forma não determinística, e se nenhuma for verdadeira, ocorre um erro de execução, o que obriga o programador a tratar todos os casos. No exemplo do máximo, quando x é igual a y, as duas guardas são verdadeiras, mas tanto faz qual ramo executa, pois o resultado é o mesmo. Assim, o código expressa apenas o que importa para o problema, sem impor uma ordem desnecessária entre os casos, o que facilita provar que o programa está correto.

O laço "do ... od" segue a mesma ideia: ele se repete enquanto alguma guarda for verdadeira, escolhendo uma ao acaso a cada volta, e termina quando nenhuma for. Com isso, o caminho percorrido pode mudar a cada execução, mas o resultado final continua garantido. Os comandos protegidos serviram de base para o CSP de Hoare (1978), e o "select" de Go é um descendente direto: quando vários canais estão prontos ao mesmo tempo, o Go sorteia qual deles será atendido. Já as guardas de Haskell herdaram o nome e a sintaxe, mas são avaliadas em ordem, sendo portanto determinísticas.

Evidências:

a) Go (select.go):
print 1ª execução:



```
1 package main
2
3 import "fmt"
4
5 func main() {
6     a := make(chan string, 1)
7     b := make(chan string, 1)
8     contagem := map[string]int{}
9
10    for i := 0; i < 1000; i++ {
11        a <- "a"
12        b <- "b"
13
14        select {
15            case x := <-a:
16                contagem[x]++
17            case x := <-b:
18                contagem[x]++
19        }
20
21        select { // esvazia o canal que nao foi escolhido
22            case <-a:
23            case <-b:
24        }
25    }
26    fmt.Println(contagem)
27 }
```

Console I/O AI Agent

map[a:499 b:501]

print 2ª execução:

```
main.go X
1 package main
2
3 import "fmt"
4
5 func main() {
6     a := make(chan string, 1)
7     b := make(chan string, 1)
8     contagem := map[string]int{}
9
10    for i := 0; i < 1000; i++ {
11        a <- "a"
12        b <- "b"
13
14        select {
15            case x := <-a:
16                contagem[x]++
17            case x := <-b:
18                contagem[x]++
19        }
20
21        select { // esvazia o canal que nao foi escolhido
22            case <-a:
23            case <-b:
24        }
25    }
26    fmt.Println(contagem)
27 }
```

Console I/O AI Agent

map[a:489 b:511]

Comentários: Nestas execuções, os dois canais estavam sempre prontos, e o "select" sorteou qual deles seria lido em cada uma das 1000 repetições. A contagem ficou próxima de metade para cada canal, mas os números mudaram de uma execução para a outra, mostrando a escolha não determinística entre guardas verdadeiras.

b) Python (guardas.py):

print 1ª execução:

```
main.py X +
1 import random
2
3 q = [4, 3, 2, 1]
4 print("Inicio:", q)
5
6 while True:
7     guardas = [i for i in range(3) if q[i] > q[i + 1]]
8     if not guardas:
9         break
10    i = random.choice(guardas)
11    q[i], q[i + 1] = q[i + 1], q[i]
12    print(f"Guarda q{i+1} > q{i+2} escolhida: {q}")
13
14 print("Final:", q)
```

Console I/O AI Agent

Inicio: [4, 3, 2, 1]
Guarda q3 > q4 escolhida: [4, 3, 1, 2]
Guarda q2 > q3 escolhida: [4, 1, 3, 2]
Guarda q3 > q4 escolhida: [4, 1, 2, 3]
Guarda q1 > q2 escolhida: [1, 4, 2, 3]
Guarda q2 > q3 escolhida: [1, 2, 4, 3]
Guarda q3 > q4 escolhida: [1, 2, 3, 4]
Final: [1, 2, 3, 4]

print 2ª execução:

```
main.py X +
1 import random
2
3 q = [4, 3, 2, 1]
4 print("Inicio:", q)
5
6 while True:
7     guardas = [i for i in range(3) if q[i] > q[i + 1]]
8     if not guardas:
9         break
10    i = random.choice(guardas)
11    q[i], q[i + 1] = q[i + 1], q[i]
12    print(f"Guarda q{i+1} > q{i+2} escolhida: {q}")
13
14 print("Final:", q)
```

Console I/O AI Agent

Inicio: [4, 3, 2, 1]
Guarda q3 > q4 escolhida: [4, 3, 1, 2]
Guarda q2 > q3 escolhida: [4, 1, 3, 2]
Guarda q1 > q2 escolhida: [1, 4, 3, 2]
Guarda q2 > q3 escolhida: [1, 3, 4, 2]
Guarda q3 > q4 escolhida: [1, 3, 2, 4]
Guarda q2 > q3 escolhida: [1, 2, 3, 4]
Final: [1, 2, 3, 4]

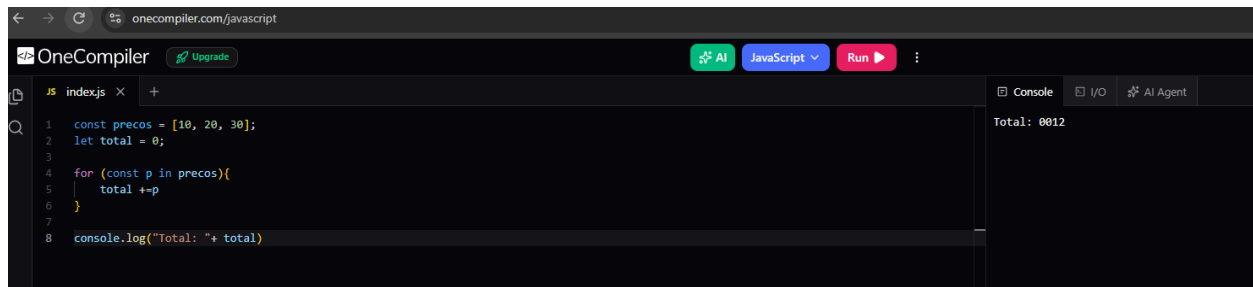
Comentários: Nestas execuções, o programa simula o laço "do ... od" do livro, escolhendo ao

acaso uma das guardas verdadeiras (um par fora de ordem) e trocando seus valores. A sequência de trocas foi diferente em cada execução, mas o laço só terminou quando nenhuma guarda era verdadeira, e por isso o resultado final foi sempre a lista ordenada.

Slide 22:

JavaScript

A) Saida:



```
1 const precos = [10, 20, 30];
2 let total = 0;
3
4 for (const p in precos){
5   total +=p
6 }
7
8 console.log("Total: "+ total)
```

Console: Total: 0012

B) O que causa o erro:

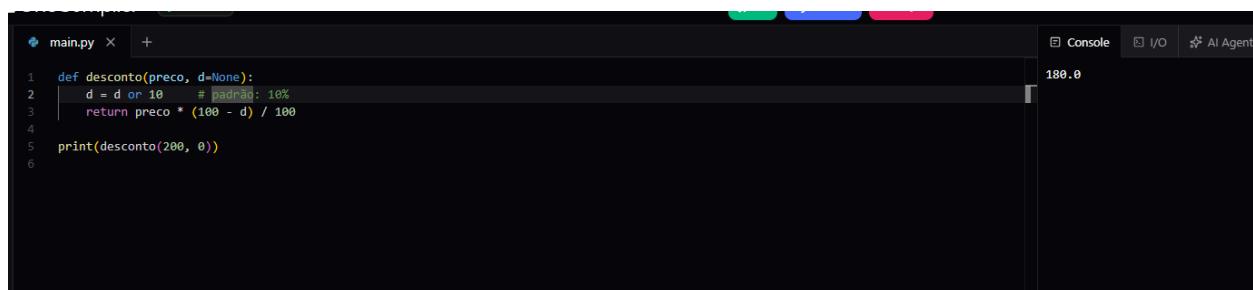
R: O erro é causado ao se utilizar o “in”, pois o in itera no total os índices de cada valor, não o seu valor de fato.

C) Como corrigir:

R: Para corrigir, é necessário primeiro trocar o operador `in` pelo `of`, pois o `of` irá acessar o valor de cada índice do array.

Python:

A) saida:



```
1 def desconto(preco, d=None):
2   d = d or 10 # padrão: 10%
3   return preco * (100 - d) / 100
4
5 print(desconto(200, 0))
6
```

Console: 180.0

B) O que causa o erro:

R: o erro é causado pelo operador “or”. Isso acontece porque, quando o or é usado em comparação entre dois valores e um deles é 0, o OR considera o 0 como uma expressão booleana. Sendo assim, o código faz a leitura de que, se o valor for falso (0), então retorna 10. Sendo assim, quando foram passados os valores 200 e 0, o sistema entendeu que era para puxar 10% de desconto.

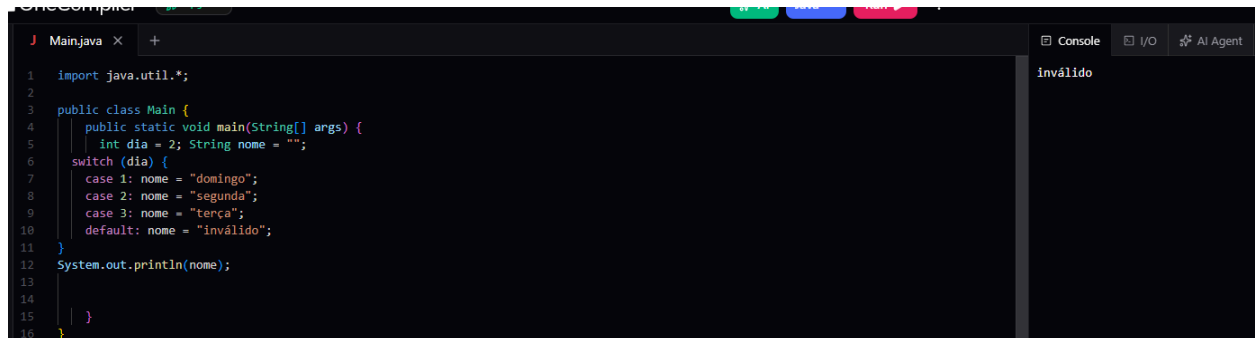
C) Como corrigir:

R: A verificação deve ser feita pelo is None`, como “if d is None”. Sendo assim, quando se passa o valor 0, o sistema identifica que não é nulo, portanto vai passar 10, e se não passar nenhum valor, o sistema vai aplicar 10% de desconto.

Slide 23:

Java;

A) Saída:



```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int dia = 2; String nome = "";
6         switch (dia) {
7             case 1: nome = "domingo";
8             case 2: nome = "segunda";
9             case 3: nome = "terça";
10            default: nome = "inválido";
11        }
12        System.out.println(nome);
13    }
14 }
15
16 }
```

Console: inválido

B) O que causou o erro:

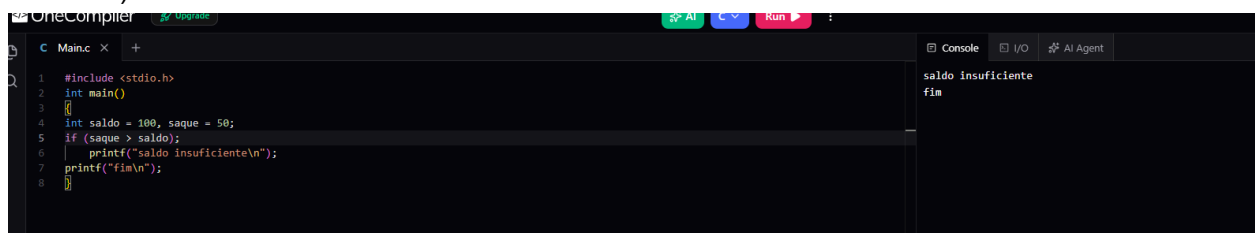
R: O que causou o erro foi a falta de break a cada case, pois o sistema, sem o break, não sabe onde parar, dessa forma ele passa a executar todo o código, seguindo para então no default nome = "inválido", atribuindo esse valor final ao nome.

C) Como corrigir:

R: Para corrigir, seria apenas informar o “break;” para cada case.

C;

A)Saída:



```
1 #include <stdio.h>
2 int main()
3 {
4     int saldo = 100, saque = 50;
5     if (saque > saldo)
6         printf("saldo insuficiente\n");
7     printf("fim\n");
8 }
```

Console: saldo insuficiente
fim

B) O que causou o erro;

R: O que causou o erro foi o uso da “,” depois do if, pois o saldo inválido não está dentro do if, porque o if termina antes de começar.

C) Como corrigir:

R: Para corrigir, seria necessário retirar o “,” e adicionar algo dentro do if.

Slide 24:

Go:

A) Saída:



```
1 package main
2 import "fmt"
3
4 func main() {
5     x := 10
6
7     if x > 5 {
8         x := x * 2
9         fmt.Println("dentro:", x)
10    }
11    fmt.Println("fora:", x)
12 }
13
```

dentro: 20
fora: 10

B) O que causou o erro:

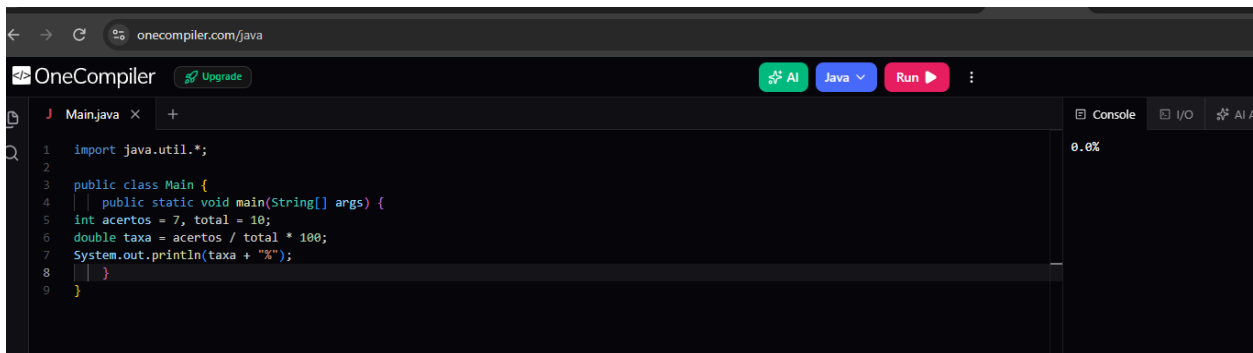
R: O operador `:=` faz com que seja criada uma nova variável em vez de se utilizar a já existente.

C) Como corrigir;

R: Para corrigir, bastaria mudar de `:=` para `=`, ficando `x = x * 2`.

Java:

A) Saída:



```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         int acertos = 7, total = 10;
6         double taxa = acertos / total * 100;
7         System.out.println(taxa + "%");
8     }
9 }
```

0.0%

B) O que causou o erro:

R: Não tem o tratamento dos números inteiros antes de passar por uma contagem a partir de um double, sendo assim o sistema calcula $7/10 = 0$, pois não tem decimais, e faz 0×100 .

C) Como corrigir:

Para corrigir, bastaria passar os acertos para double antes de começar a divisão.

`double taxa = (double) acertos / total * 100;`

Slide 25:

1) Precedência

A) O que acontece:

A conta $-2 ** 2$ dá -4 em vez de 4 . Isso porque o sinal negativo é calculado depois da potência, então fica $-(2^2) = -4$.

B) O que causou:

A regra de precedência. O operador `**` (potência) tem prioridade maior que o `-` (negativo), então a potência é feita primeiro.

C) Como corrigir:

Colocar parênteses: `(-2) ** 2`. Assim o negativo é aplicado antes da potência.

2) Operadores

A) O que acontece:

A expressão `+i + (+i)` dá 0 em vez de incrementar. Porque `+` antes de parênteses não incrementa nada, é só um operador de positivo.

B) O que causou:

O `+` não é o incremento (`++`). Sem `++`, a variável não muda, só soma o valor dela duas vezes.

C) Como corrigir:

Usar `++i` ou `i++` em vez de `+i`. Exemplo: `i++; resultado = i + i;`

3) Conversões

A) O que acontece:

A string `"1.3"` vira 1 (perde a parte decimal). A conversão automática trunca números decimais.

B) O que causou:

A conversão silenciosa de string para inteiro. A linguagem converte sem avisar e descarta os decimais.

C) Como corrigir:

Fazer a conversão explícita e correta. Em vez de `atoi()`, usar `strtod()` ou `parseFloat()` que preserva decimais.

4) Booleanos

A) O que acontece:

A string `"0"` é considerada verdadeira quando deveria ser falsa.

B) O que causou:

Em C e Python, strings não-vazias são sempre verdadeiras, mesmo que contenham `"0"`. A linguagem não diferencia a string `"0"` do número 0.

C) Como corrigir:

Ser explícito: fazer uma comparação clara em vez de converter direto. Exemplo: `if (str != "")` em vez de apenas `if (str)`.

5) Atribuição**A) O que acontece:**

`if (x = y)` atribui `y` em `x` em vez de comparar. Não testa igualdade, só coloca o valor.

B) O que causou:

`=` é atribuição, não comparação. É fácil confundir com `==` (que compara).

C) Como corrigir:

Usar `==` para comparar: `if (x == y)`. Sempre usar `==` no `if`.

6) Seleção (Switch)**A) O que acontece:**

Sem `break`, o `switch` executa vários cases seguidos ("fall-through"). Imprime mais coisas do que esperado.

B) O que causou:

Sem `break`, a execução não para após um case e continua pelos próximos.

C) Como corrigir:

Colocar `break`; ao final de cada case. Assim só executa o que você quer.

7) Laços**A) O que acontece:**

O laço não incrementa e fica infinito, ou nunca executa se a condição for falsa.

B) O que causou:

Falta de incremento (`i++`) ou inicialização errada no `for`.

C) Como corrigir:

Usar a sintaxe correta: `for (int i = 1; i <= 10; i++)`. Sempre colocar a inicialização, condição e incremento.

8) Desvio

A) O que acontece:

goto ou return desvia o fluxo de forma confusa e o código após não executa.

B) O que causou:

goto permite desvios aleatórios e return sai da função na hora. Fica código desorganizado.

C) Como corrigir:

Evitar goto. Usar if/else, while, break, continue. Estruturas claras e ordenadas.

Slide 26:

A) O que é diferente:

Em Ruby, operadores como + e * parecem especiais, mas são na verdade métodos. Quando você escreve `1 + 2`, Ruby traduz para `1.+(2)`. Em Lisp, tudo é explicitamente uma função em notação prefixa: `(+ 1 2)`.

B) O que causou a diferença:

Ruby quer parecer natural e legível como matemática comum. Lisp prioriza simplicidade: sem precedência de operadores, os parênteses deixam claro o que fazer primeiro.

C) Como usar cada uma:

Ruby: escreva como de costume `(1 + 2)`. Lisp: sempre prefixa `(+ 1 2)`. Para somar vários números em Lisp, fica `(+ 1 2 3 4 5)` em uma só função.

Conclusão: Ruby esconde que + é método para ser legível. Lisp expõe tudo. Ruby parece mais natural, Lisp é mais lógico.